

Performance Profiling Report — Address Endpoint

Author: Drew Date: 2026-04-30

Setup

Profiled POST /address/request (and /address/count), which calls AddressService.request() → fetch('https://address.nerdstacks.org/'). Spec named ischool.gccis.rit.edu/addresses/, but that host is currently 500-ing; the deployed service points at nerdstacks with the same schema. Server run with node --prof --inspect. Workload: {"zipcode": "10001"} → upstream returns **1,736 records / ~392 KB JSON**.

Results

Latency (curl, 5 runs each):

Endpoint	Cold	Warm avg
/address/count	276 ms	75 ms
/address/request	75 ms	72 ms

V8 tick report (27,226 samples): 99.4% ntdll.dll (kernel I/O wait), 0.2% JavaScript total — and most of that JS is the ts-node compiler at startup. Zero hot frames in our code.

Memory (harness, 5 sequential calls, post-GC baseline 186 MB RSS / 107 MB heap):

	RSS Δ	heapUsed Δ	external Δ
After call 5	+14.39 MB	+11.24 MB	+7.00 MB
After manual GC	+6.95 MB	+2.79 MB	+7.00 MB (not freed)
One response retained	+5.60 MB	+3.15 MB	+2.52 MB

Holding one 1,736-record response = **~5–6 MB**. A 14623 query (12,202 records) extrapolates to ~35–40 MB.

Why

- The bottleneck is the WAN round-trip to the upstream**, not our code. 99% of process time is blocked on a socket. Express routing, executeSubRoute, JSON.parse, and responseWrapper together are statistical noise.
- await res.json() (address.service.ts:43) **buffers the full body and allocates a 1,736-element array**. count() immediately throws all of it away except .length — we pay full parse+allocation to read a number.
- /count **and** /request **cost the same** (75 ms vs 72 ms warm) because the count path does the identical fetch+parse; the only saving is in the response sent back to the client.
- external **memory doesn't return on GC** (+7 MB persists). Under sustained traffic this is RSS creep invisible to old-space limits.
- distance() (address.service.ts:60) **fetches the full N-record payload, then pop().pop()s the last two** — same allocation cost for 0.1% of the useful output.

Future Changes

#	Change	Why
1	Cache responses by zipcode (short TTL, in-memory LRU).	Warm calls 70 ms → <2 ms; we already hit 503 rate-limits during this run, caching removes that pressure entirely.
2	Stream-count instead of await res.json() in count() (e.g. stream-json).	Drops per-call heap from ~5 MB to <100 KB; no behaviour change.
3	Enable HTTP keep-alive on the upstream fetch (custom Agent with keepAlive: true).	Saves 30–50 ms on cold-after-idle TLS resumption.
4	distance() should request only the two records it needs , not the full list.	Removes a ~5 MB allocation per /address/distance.
5	Run production builds (npm run build → node dist/server.js) instead of ts-node.	Removes TS compiler from the hot path; smaller startup + memory footprint.

Bottom Line

The endpoint is **I/O-bound and allocation-heavy, not CPU-bound**. Highest-leverage latency fix is **caching** (~35× warm-path improvement). Highest-leverage memory fix is **streaming the count path** (eliminates the per-call multi-MB allocation that GC only partially reclaims).